PROJECT 3 · PART B

Flow Matching from Scratch

Training U-Nets to remove noise, learn a time-dependent flow, and generate handwritten digits.

Part A · Exploring flow models **Part B · Flow matching from scratch**

Part A used a pretrained image model. Here I build and train the denoisers and flow models themselves on MNIST. The first experiments show what a one-step network learns from corrupted digits; the second teaches a U-Net how to move a sample from random noise to the data distribution.

PART 1 · SINGLE-STEP DENOISING

1.1 Building the denoising U-Net

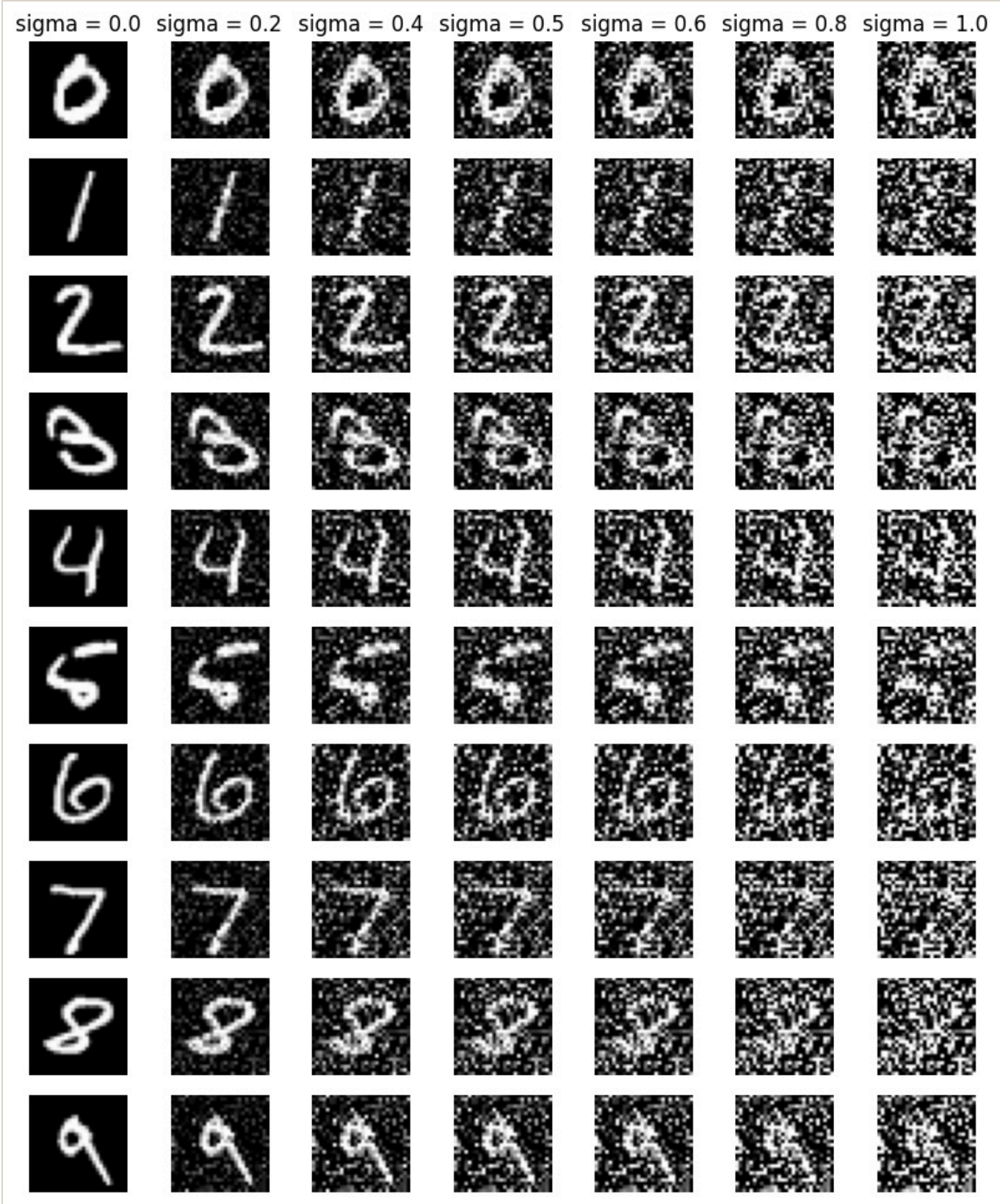
I implement the image-to-image model as a U-Net: convolutions learn local features, downsampling adds context, and upsampling restores the 28 × 28 resolution. Skip connections join encoder and decoder features at matching scales, preserving fine digit strokes that could be lost in the bottleneck. The provided blocks use convolution, batch normalization, and GELU; the output layer predicts one grayscale image.

PART 1 · SINGLE-STEP DENOISING

1.2 Constructing noisy MNIST inputs

MNIST images are normalized to [0, 1]. I add independent standard Gaussian noise at each prescribed strength. Holding the example and noise realization fixed across columns makes it possible to see how increasing σ changes the same digit.

$$z = x + \sigma\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$



Noising schedule · σ = 0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0

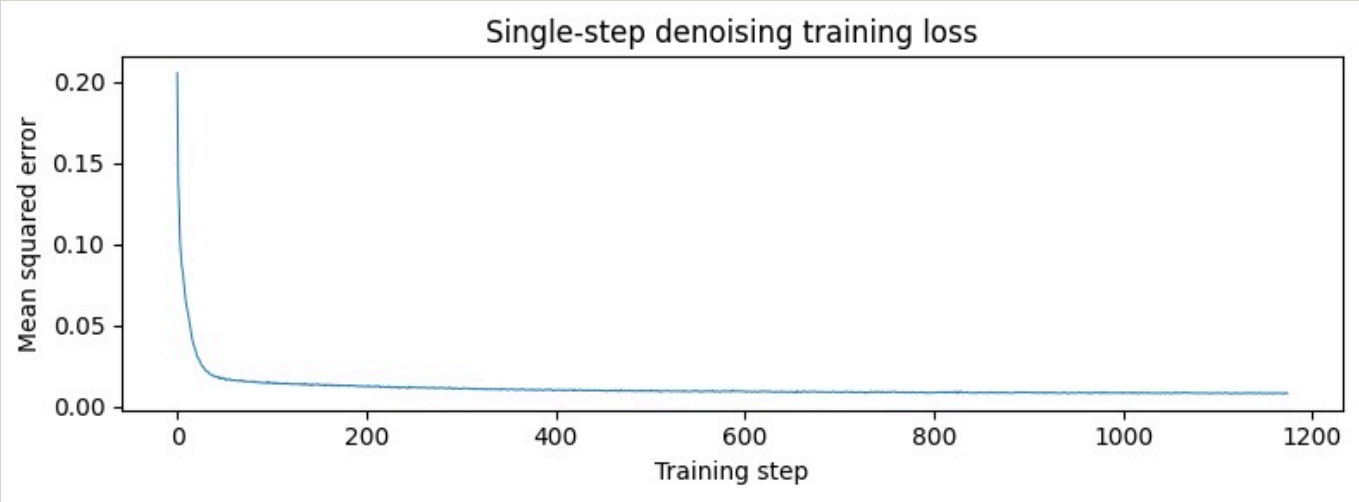
PART 1 · SINGLE-STEP DENOISING

1.2.1 Training a single-step denoiser

The unconditional U-Net is trained to map a noisy digit directly to its clean target. Each batch receives fresh σ = 0.5 noise, so the model sees different corruptions over training. I use the assignment's five-epoch setup and compare the same held-out digits and noise at epochs 1 and 5.

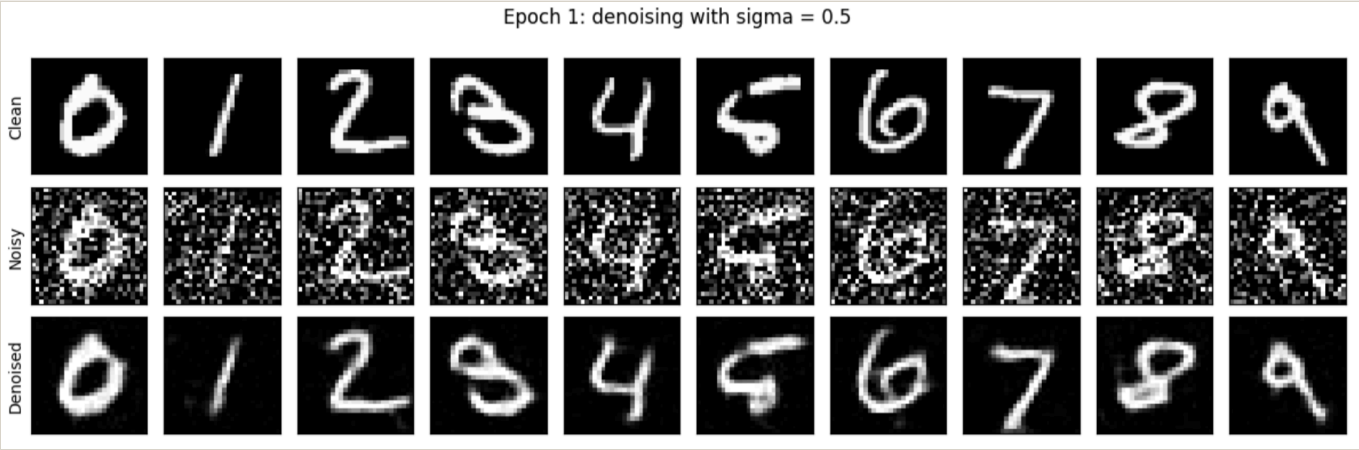
$$L = \mathbb{E}[\|D_\theta(x + 0.5\varepsilon) - x\|^2]$$

Training loss



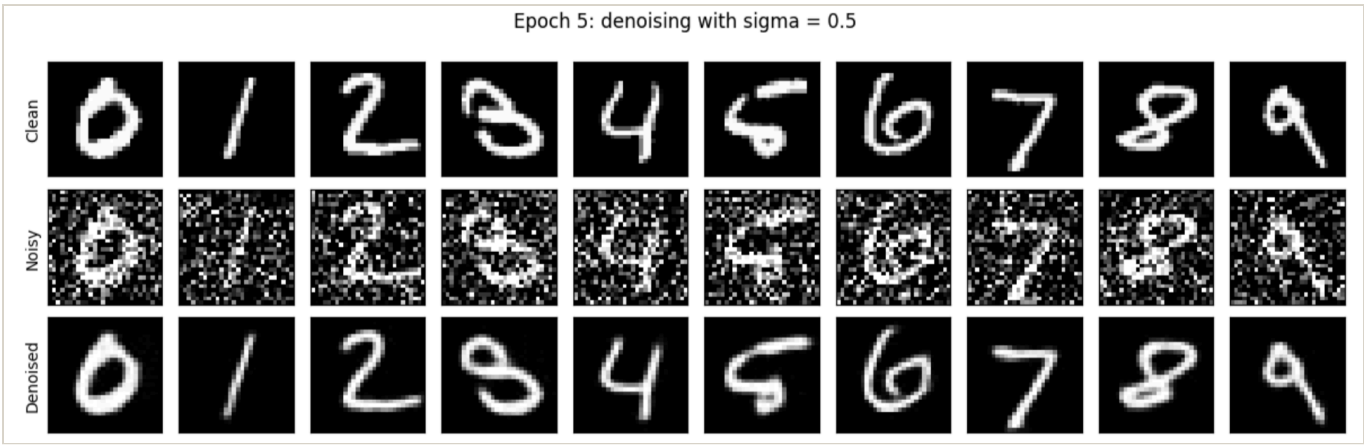
Single-step denoiser · Training loss across five epochs

Epoch 1



Clean, noisy, and denoised test digits after epoch 1

Epoch 5



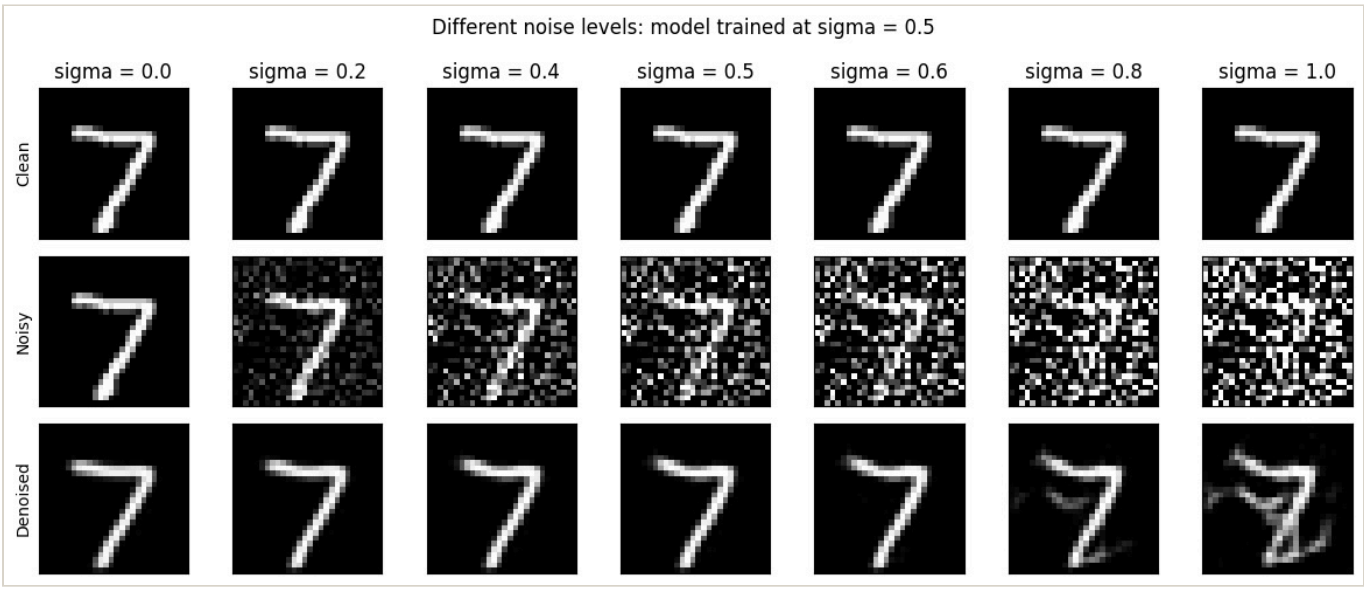
The same test digits after epoch 5

By epoch 1, the model recovers the broad shape of most digits. By epoch 5, the estimates are cleaner and closer to the clean row, while retaining some softened or altered strokes. Evaluating identical examples at both checkpoints makes this change easier to attribute to training.

PART 1 · SINGLE-STEP DENOISING

1.2.2 Testing at unseen noise levels

The denoiser was trained with σ = 0.5. To check how it handles inputs outside that training condition, I hold one test digit and its noise pattern fixed, vary σ, and apply the same trained model to each version.



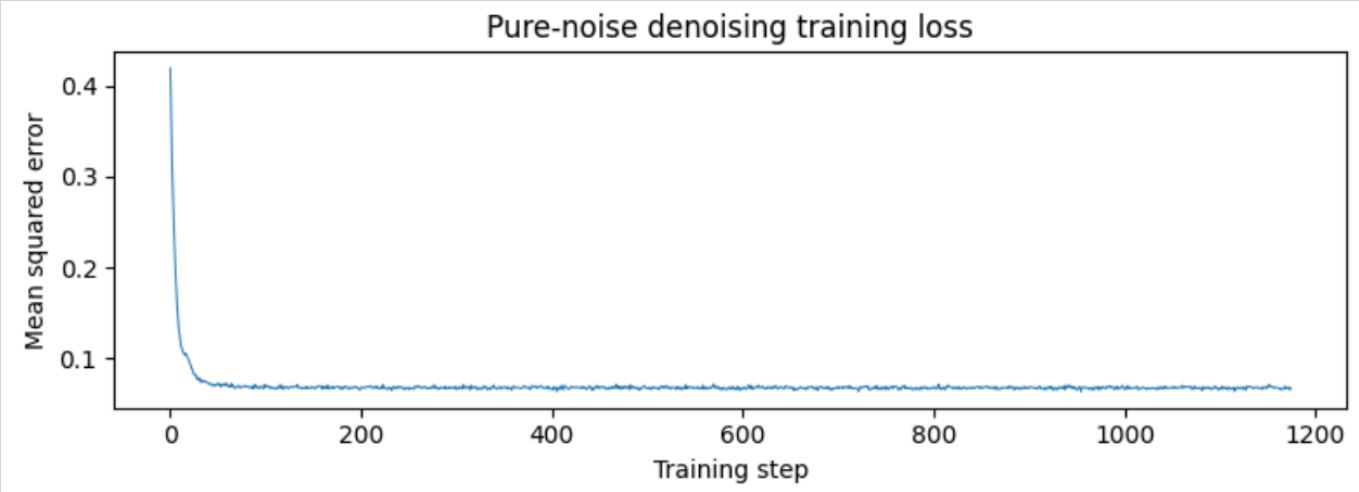
One held-out 7 · σ = 0 to 1.0

At lower noise levels, the digit stays legible with only a small perturbation. At high σ, the same 7 is harder to recover; the strongest-noise estimates become less faithful and develop extra marks. This illustrates the limit of a model trained on one noise strength when the input corruption changes.

1.2.3 Learning to denoise pure noise

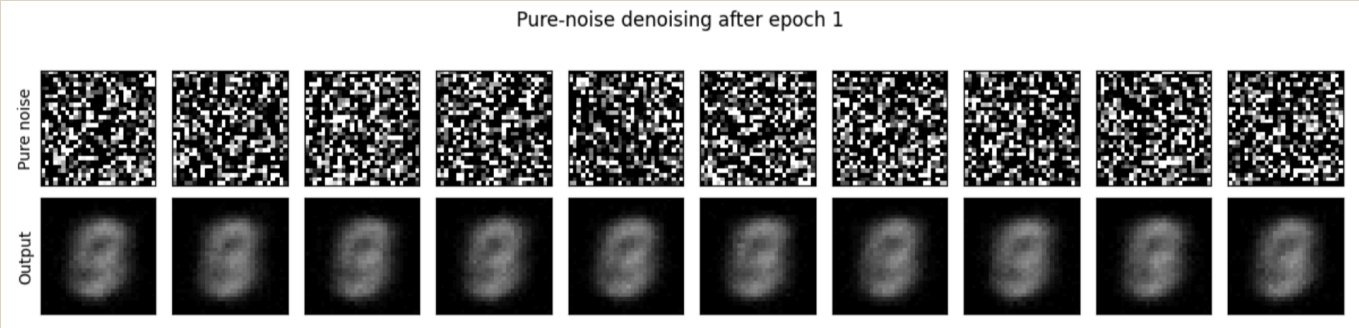
For this experiment the input is independent Gaussian noise, so it contains no information about the clean target digit. The U-Net is otherwise trained against clean MNIST images with the same mean-squared-error objective. The noise is resampled during training, and the displayed test inputs stay fixed between epochs.

$$L = \mathbb{E}_{x,\varepsilon}[\|D_\theta(\varepsilon) - x\|^2]$$



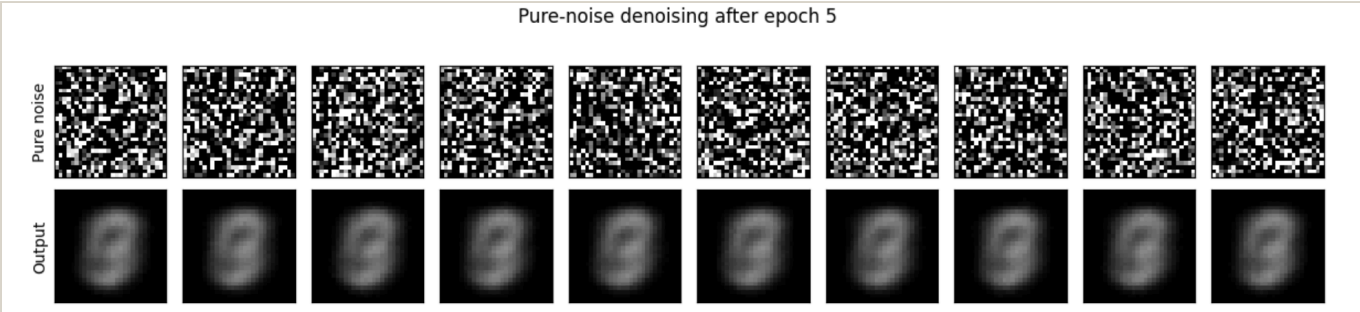
Pure-noise denoiser · Training loss across five epochs

Epoch 1



Ten pure-noise inputs and their outputs after epoch 1

Epoch 5



The same pure-noise inputs after epoch 5

The outputs all approach a similar blurry, “3”-like average instead of following their different noise patterns. Since noise and target are independent, no observation tells the model which digit to reproduce. Under squared error, the best prediction is the pixelwise mean of the training digits; blending their strokes produces a soft, ambiguous shape. This matches the repeated pattern in both epochs.

PART 2 · FLOW MATCHING

2.1 Conditioning a U-Net on time

Flow matching trains a network to predict how an image should move from Gaussian noise toward a data example. A random MNIST image x_1 and noise x_0 define points along a straight interpolation; the target velocity is their constant difference. The model must see t because the same intermediate image statistics call for different predictions at different points along the path.

$$x_t = (1 - t)x_0 + tx_1, \quad u = x_1 - x_0$$

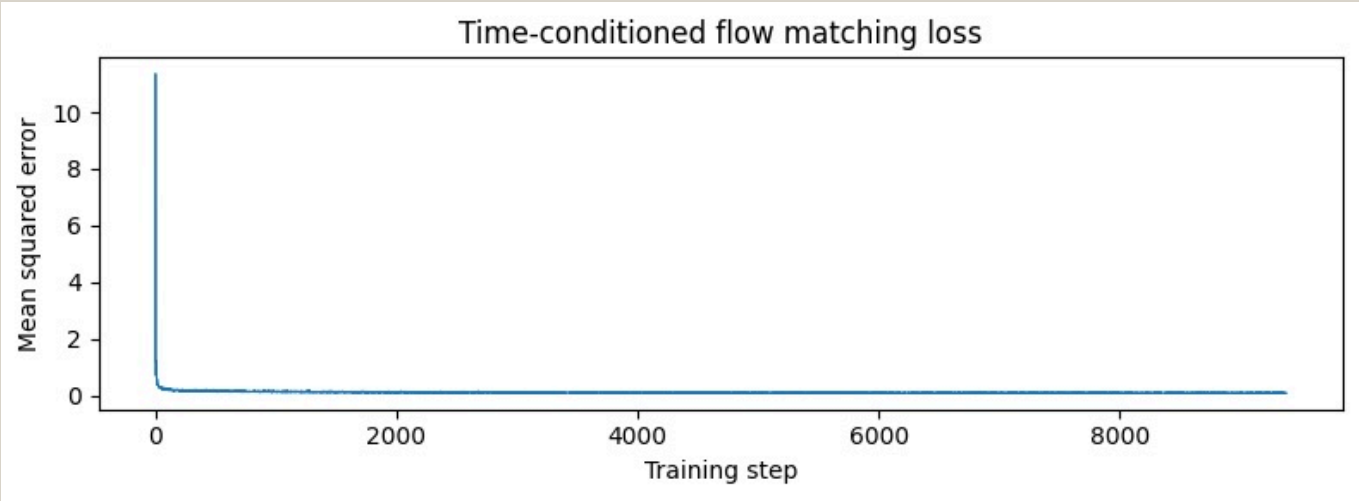
A small fully connected block embeds the scalar time value. Its features modulate internal U-Net activations, while image skip connections continue to carry spatial detail. Training uses a mean-squared error between predicted and target velocity. At generation time, Euler steps repeatedly add the predicted velocity, updating the image and time together.

PART 2 · FLOW MATCHING

2.2 Training the time-conditioned flow model

The time-conditioned U-Net is trained on shuffled MNIST batches with random Gaussian starting points and random times. Each training example is interpolated between noise and a clean digit, and the model predicts $x_1 - x_0$. The run shown uses the assignment’s ten-epoch schedule, with an exponential learning-rate decay.

$$L = \mathbb{E}_{x0,x1,t}[\|u_\theta(x_t, t) - (x1 - x0)\|^2]$$



Time-conditioned flow matching · Training loss

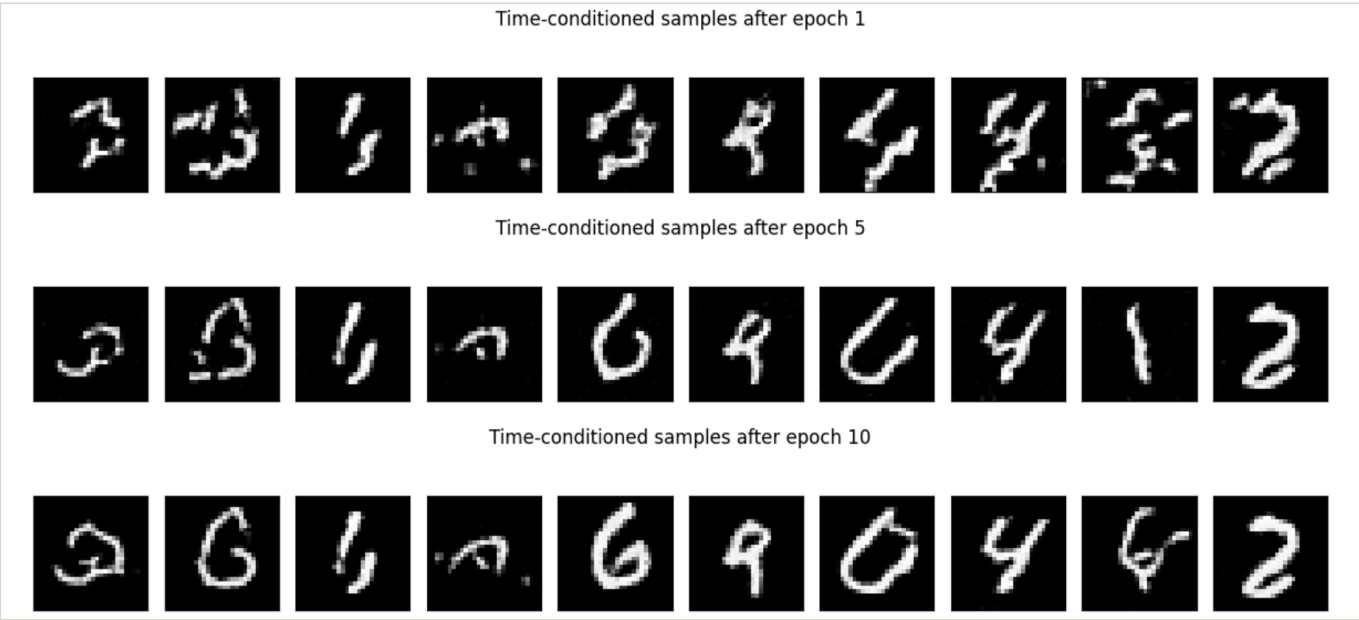
The loss falls sharply during the early updates, then changes more gradually as training continues. The curve tracks velocity-prediction error on training interpolants; sample grids are a separate visual check of whether integrating that field produces legible digits.

PART 2 · FLOW MATCHING

2.3 Sampling from the time-only model

Sampling starts from a fresh noise image at t = 0. I divide the interval into 50 Euler steps and repeatedly evaluate the time-conditioned network, moving each state toward t = 1. The grid shows ten independently seeded samples from each saved checkpoint.

$$x_{t+h} = x_t + h\, u_\theta(x_t, t), \quad h = 1/50$$



Time-only samples · Epochs 1, 5, and 10 · Ten seeds per checkpoint

Epoch 1 mostly produces fragmented strokes. By epochs 5 and 10, more samples have connected loops and recognizable digit-like structure, though several images remain malformed or ambiguous. Training improves the learned flow, but this unconditional model has no class label to control which digit should appear.

PART 2 · FLOW MATCHING

2.4 Class-conditioned U-Net

The class-conditioned U-Net receives both the current time and a one-hot digit label. Separate learned embeddings modulate decoder features so the model can steer its flow toward a requested class. During training, the label is dropped for 10% of examples; this teaches the same model to make unconditional predictions for classifier-free guidance.

$$v_{\text{guided}} = v_u + 5(v_c - v_u)$$

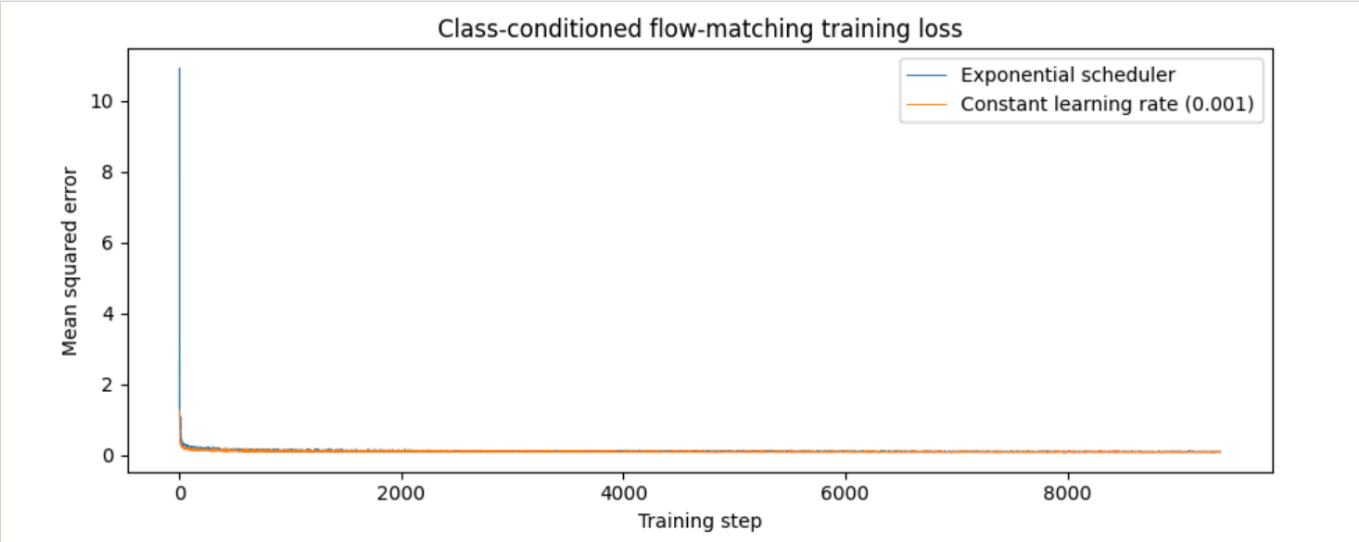
Section 2.4 develops the architecture used in the following experiments; the assignment lists no separate result deliverable for this section.

PART 2 · FLOW MATCHING

2.5 Training the class-conditioned models

Both models use the class-conditioned U-Net with hidden dimension 64 and 10% class dropout. The scheduled run uses Adam, starts at learning rate 0.01, and decays exponentially for ten epochs. The independent comparison uses a constant learning rate of 0.001, matching the scheduled run's rate at the end of training.

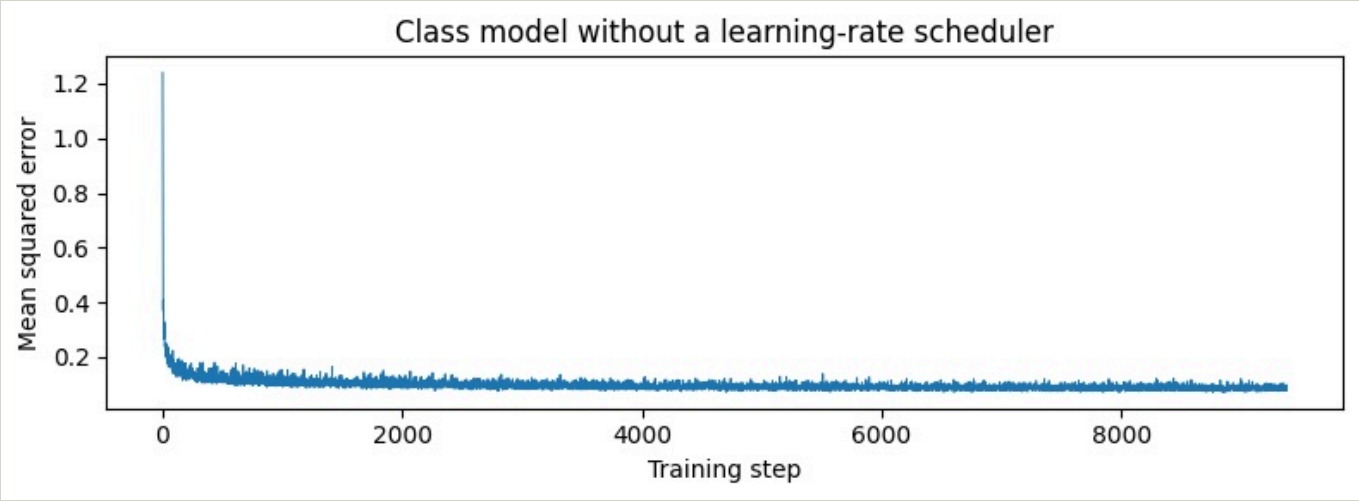
Scheduled and constant-rate comparison



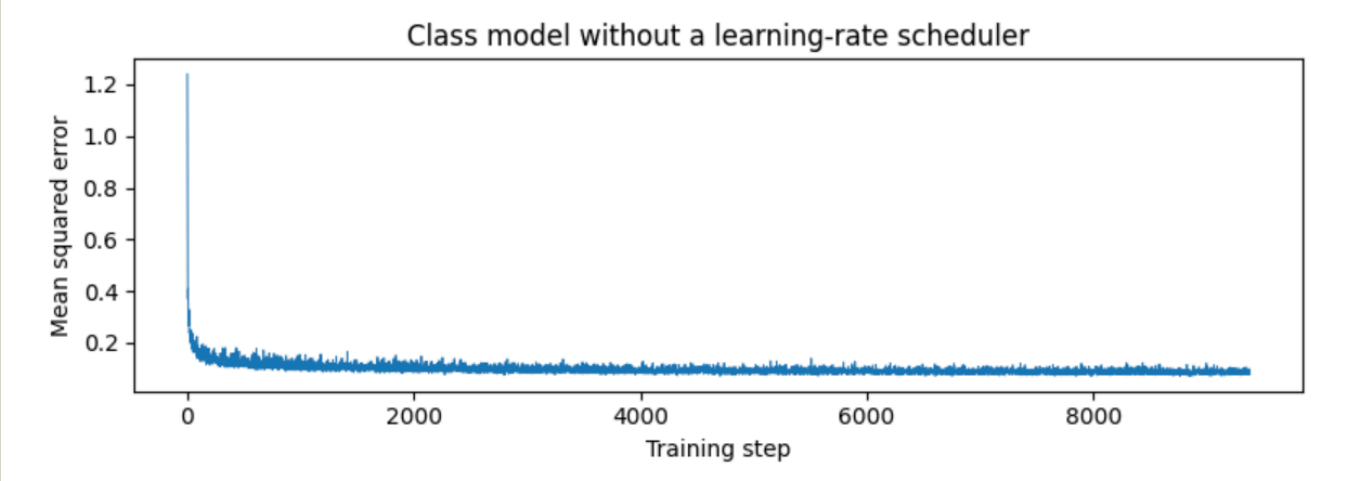
Class-conditioned training loss · Exponential scheduler and constant learning rate 0.001

Both losses fall sharply at the start and continue to decrease. The traces remain close later in training, suggesting the smaller constant rate maintains a similar training loss over this run.

Constant-rate run



No-scheduler model · Training loss



No-scheduler run · Additional loss plot

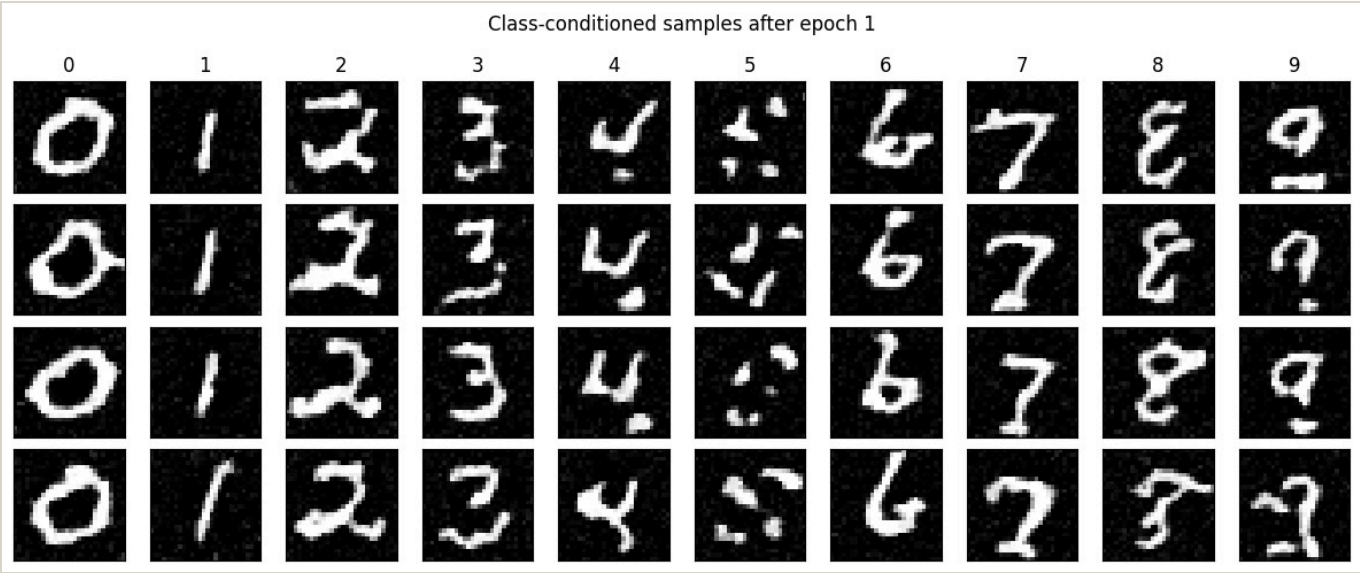
The standalone curves make the noisy step-to-step loss visible after the sharp initial drop. The learning rate was chosen to match the scheduled model's final rate, providing a direct and simple scheduler comparison.

PART 2 · FLOW MATCHING

2.6 Class-conditioned sampling

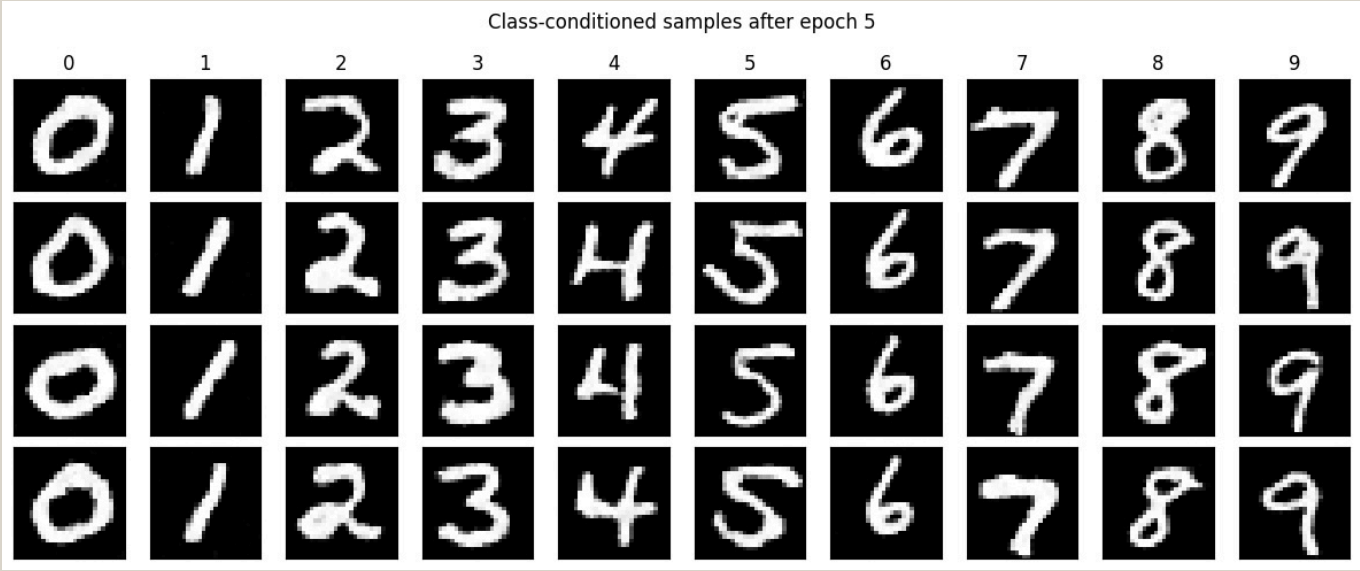
For each sample grid, the model starts from random noise and takes 300 Euler updates. Classifier-free guidance uses $\gamma = 5$. Each grid has four rows; each row includes one sample for every digit from 0 to 9. I reuse the same seed across checkpoints and models so corresponding cells start from identical noise.

Epoch 1



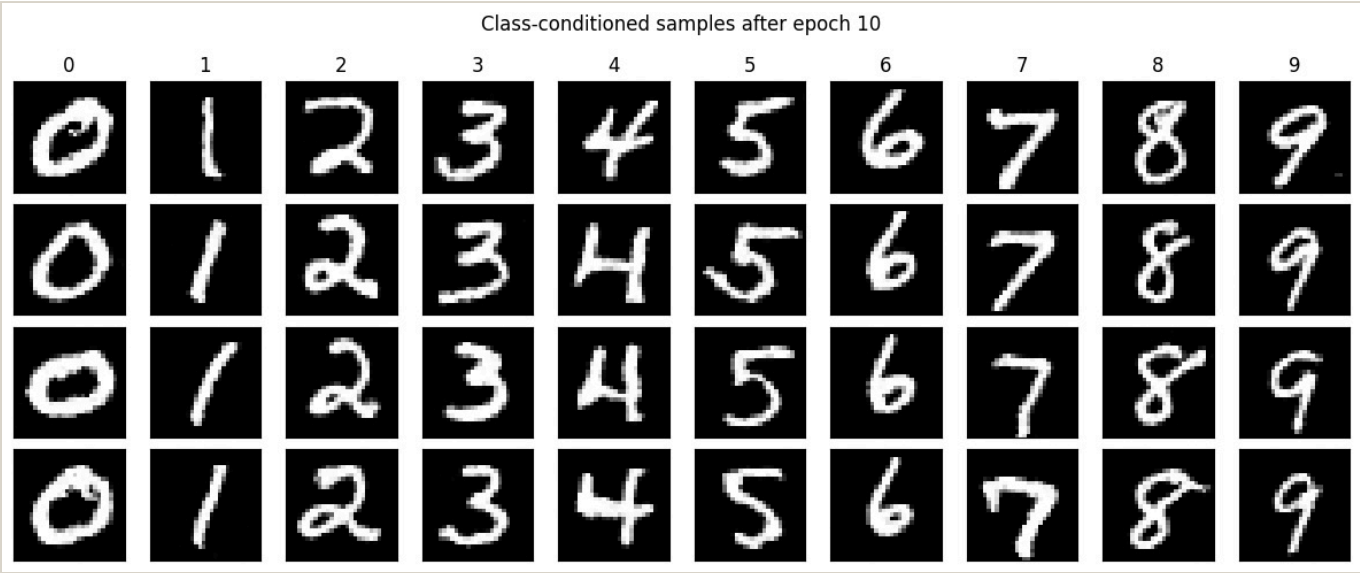
Four samples per digit after epoch 1

Epoch 5



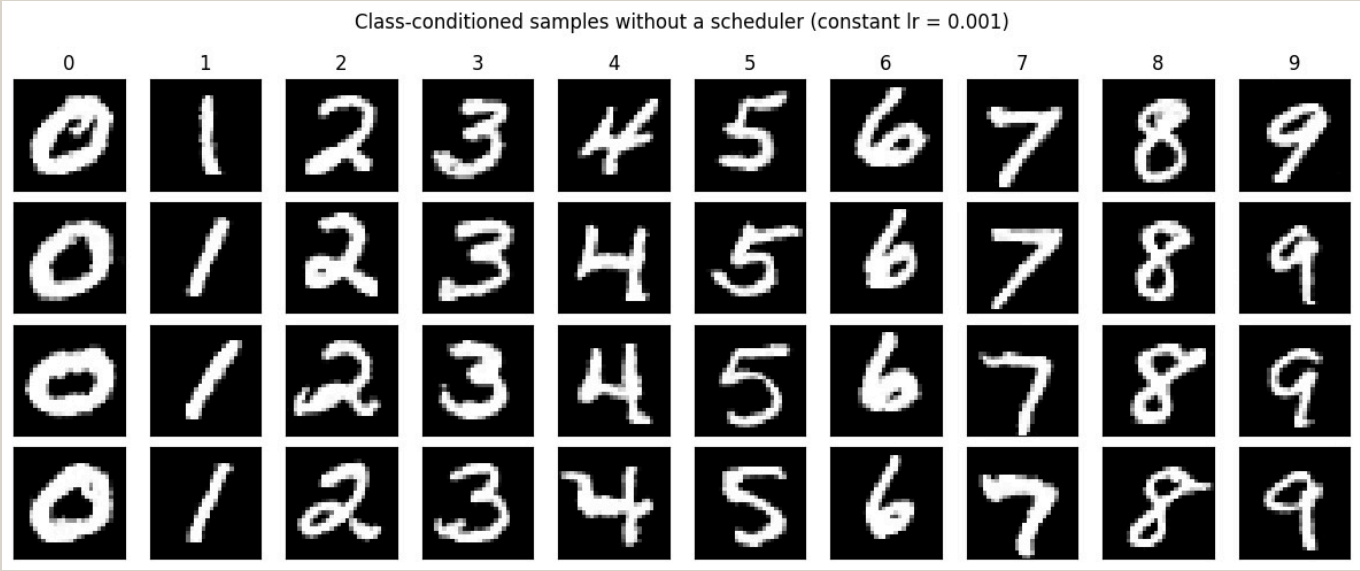
Four samples per digit after epoch 5

Epoch 10



Four samples per digit after epoch 10

Without the scheduler



Four samples per digit · Constant learning rate 0.001

At epoch 1, several classes already have recognizable forms, though some digits are malformed. By epochs 5 and 10, the requested labels are clear across nearly every row. The constant-rate model also produces legible examples that look broadly similar to the scheduled epoch 10 grid; differences remain in individual stroke shapes. The smaller constant rate therefore preserves useful sample quality here without an exponential schedule.

SUBMISSION CHECKLIST

Part B results

The current results include the denoising and time-only experiments, the class-conditioned training comparison, and guided samples for epochs 1, 5, and 10 with the no-scheduler model. Section 2.4 is represented by the implemented architecture and does not require a separate deliverable.

Download the student notebook or website PDF. The assignment requires the code and website PDF on Gradescope, then the website URL in the project Google Form once both parts are complete.